

PROJECT AND TEAM INFORMATION

Project Title

Virtual File System

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>Rootrix</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Naithani, Tanish –</i> <i>25100011477</i> <i>25100011477.geu.ac.in</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Singh, Tushar –</i> <i>2510011704</i> <i>2510011704@geu.ac.in</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Tayal, Rounak –</i> <i>2510370114</i> <i>2510370114@geu.ac.in</i> 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	<i>Rawal, Aditya –</i> <i>2510380204</i> <i>2510380204@geu.ac.in</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

We use files and folders almost every day on our computers, but most of the time we do not think about what happens in the background when a file is created, deleted, moved, or modified. There's a lot going on behind the scenes when a computer deals with files where each one actually gets stored, how directories keep everything organized, how much space is left, what metadata is attached to a file, who's even allowed to open it. Most of us never think about any of this because the operating system just handles it quietly in the background.

That's actually what got us interested in this project. We wanted to build a small Virtual File System in C++ essentially our own file system that lives entirely inside our program, where we're the ones managing the files, directories and storage blocks, not the OS. It doesn't touch or depend on the computer's real file structure at all. By building each of these pieces ourselves instead of just using them the way everyone normally does, we actually end up understanding how a file system works under the hood, not just how to use one.

The other big reason we picked this project is that it lets us actually apply the Data Structures and OOP concepts we've been studying, instead of just reading about them. Folders and subfolders naturally form a tree, so that's where a tree structure comes in. Need to find a file or directory quickly? That's where hash tables help. Keeping track of which storage blocks are free and which are taken a bitmap handles that. And to stop the whole thing from turning into a mess, we're splitting everything into classes, each one responsible for its own part of the system.

Along the way, we also get to work with block allocation, file I/O, caching, LRU replacement, journaling, snapshots, and filesystem checking concepts that sound abstract in a textbook but start making a lot more sense once you're actually implementing them and watching how a file system handles different operations under the hood.

At the end of the day, that's really the point of this project taking what we've learned in theory and actually building something with it. The command-line interface lets a user run different file and directory operations and see exactly how the virtual file system responds, which makes it a practical way to understand Data Structures and basic Operating System ideas together, not as two separate subjects.

State of the Art / Current solution (1 pt)

At present this work is mainly handled by operating systems such as Windows and Linux. When we create, open, edit or delete a file the operating system takes care of everything behind the scenes. The operating system manages folders, storage space, file locations, permissions and other details that are needed to keep files properly organized.

Operating systems also have their ways of managing storage blocks and keeping track of the space that is available. They use techniques such as caching and recovery methods. These techniques make file system operations faster. Reduce the chances of losing or damaging data if something goes wrong.

The main issue is that most of this process is not visible, to the user. We can easily. Manage files and folders but we normally cannot see how the data is actually stored or how the available storage blocks are managed internally.

To understand these processes in a way our project will create a small virtual file system using C++. The virtual file system is not meant to replace the file system of an operating system. Instead the file system will give us an environment where we can implement and observe basic file system operations ourselves. I feel that this will help us. This will help us better understand the concepts we have studied in Data Structures, OOP and Operating Systems.

Project Goals and Milestones (2 pts)

Project Goals

The main goal of this project is to design and implement a working Virtual File System or VFS using C++. This project basically gives the reader a hands-on look at how files, directories, storage blocks and file details are actually managed under the hood — using data structures, OOP, and some basic operating-system ideas along the way.

Here's what we're aiming to build:

- A disk made up of fixed-size storage blocks*
- Files and directories organized in a tree structure*
- The usual file operations — create, read, write, append, truncate, rename, copy, move, delete*
- Important file details stored properly — size, permissions, timestamps, block locations*
- A bitmap to track which blocks are free and which are occupied, along with block allocation*
- The ability to search for files by name, path or other metadata*
- A cache for blocks that get accessed frequently*
- Extra features like journaling, filesystem checking and snapshots*

A command-line interface to tie it all together and let users actually interact with the system

- Allow users to search for files using names, paths and available metadata.*
- Use a cache for frequently accessed blocks.*
- Add features such as journaling, filesystem checking and snapshots.*
- Provide a command-line interface through which users can interact with the system.*

1. System Design Before writing any code, we'll map out the overall architecture deciding which C++ classes and modules the system actually needs and how they'll fit together.

2. Virtual Disk and Block Management Build the virtual disk itself, break it into fixed-size blocks, and implement bitmap tracking so the system always knows which blocks are free and which are in use.

3. File and Directory Management Set up the directory hierarchy, then implement the core operations creating, deleting, renaming, copying and moving files and directories.

4. File Operations and Metadata Add read, write, append and truncate support, and make sure every file correctly tracks its permissions, timestamps, size and block allocation.

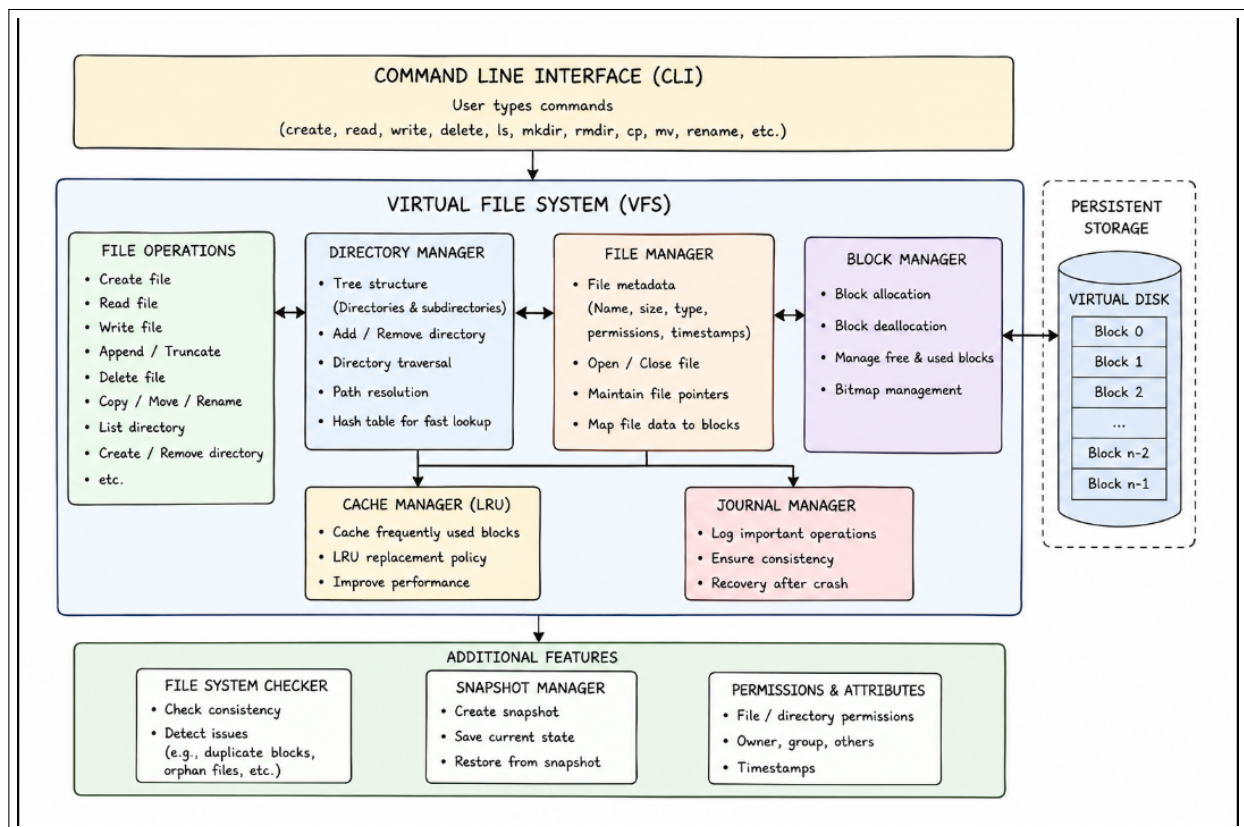
5. Advanced Features This is where it gets interesting: file search, an LRU cache, journaling, filesystem checks and snapshot support.

6. CLI, Testing and Integration Bring everything together through the command-line interface, then test end-to-end to confirm all the modules actually work as one system.

Project Approach (3 pts)

We have planned to develop this project in C++ programming using Object Oriented Programming principles. We will break the project into parts so that each part can be developed and tested separately. The parts that will comprise the project will have the disk, files, directories, block manager, cache, journal and command line interface. The virtual disk will be made of blocks all the same size. We'll use a bit map to keep track of which blocks are free and which blocks are already in use. The block manager will manage allocation and release of these blocks whenever files are created, modified or deleted. Directories will follow a tree layouts so that a directory can hold both files and other directories. When quick searching or lookup of files and directories is needed, we will use hash tables. Each file object will contain information such as the filename, file size, file type, file permissions, timestamps and the blocks where the file data is stored. The interface will enable users to perform file operations such as create files read files write to files append to files truncate files copy files move files rename files and delete files. The feature will use an LRU-based cache to manage accessed blocks. The system will be able to deal with operations, and there will be a journal to keep track of operations. Filesystem checking will be used to find issues like block problems or duplicate block allocation. Snapshots will also be used to save and restore the state of the file system. The project will primarily use C++ libraries. File I/O for storing data test the system and keep persistence

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

The expected outcome of this project is a working command-line based Virtual File System developed in C++. It will provide users with a simple environment where they can create and manage virtual files and directories without making changes to the actual file system of the computer.

The project is expected to include:

- A virtual disk divided into fixed-size storage blocks.*
- A tree-based structure for managing files and directories.*
- Basic file operations such as create, read, write, append, truncate, copy, move, rename, and delete.*
- File metadata and permission management.*
- A bitmap for keeping track of free and occupied blocks.*
- Search options for files and directories.*
- An LRU-based cache for frequently used blocks.*
- Journaling with a basic recovery mechanism.*
- Filesystem consistency checking.*
- Options to create and restore snapshots.*
- A command-line interface for accessing the different operations.*
- Basic information about disk usage and system performance.*

Through this project, we will be able to apply concepts such as trees, hash tables, bitmaps, linked structures, OOP, file I/O, memory management, caching, and basic operating-system concepts in one practical system.

Assumptions

The project will use a disk that has a fixed amount of storage. The virtual disk will be split into blocks each block having the same size. These blocks will hold the data for files.

The project will. Manage files and directories inside the virtual file system. The virtual file system will keep information for each file, such as its size, its permissions, its timestamps and where its data is stored. Users will use an interface built in C++ to run all the operations.

The project is mainly being developed so that the team can learn file system concepts. While building the project the team will use ideas, like directory trees, block allocation, metadata management, caching, journaling and filesystem checking. Will see how all these parts fit together into a single system.

References

- 1. Balagurusamy, E. Object Oriented Programming with C++. McGraw Hill Education. This book helped me learn about classes, objects, inheritance and other Object-Oriented Programming ideas that I used in the project.*
- 2. Kanetkar, Y. Data Structures Through C. BPB Publications. This book guided me on trees, linked lists, hashing and how to manage memory.*
- 3. GeeksforGeeks. Data Structure and Algorithms. This website helped me grasp data structures and how they can be used. Available at: <https://www.geeksforgeeks.org/data-structures/>*
- 4. Programiz. C++ Programming. This site served as a guide for C++ concepts, file handling and Object-Oriented Programming. Available, at: <https://www.programiz.com/cpp-programming>*